

Data Structures

Lecture 1

Introduction to Data Structures



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- **Introduction to Data Structures**
 - **Data Structures**
 - **Algorithms**
 - **The Phases of Software Development**

Data Structures

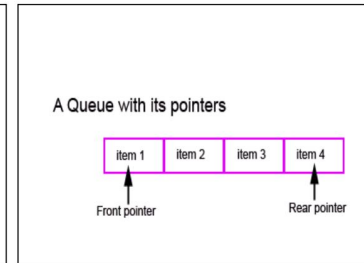
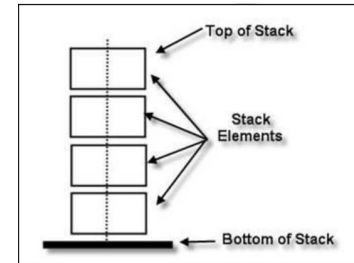
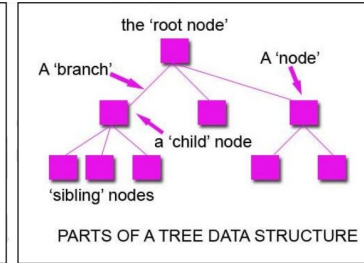
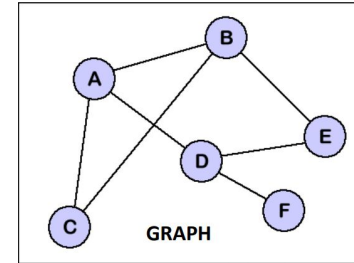
Introduction to Data Structures

A **data structure** is a specialized format for organizing, processing, retrieving and storing data. For example Sequences of numbers or tables of data.

Criteria to choose correct Data Structure:

There are several types of data structures for a specific purpose:

- Size of data.
- Way that data will be used.
- How often the data will be changed.
- Required capacity.
- The programming language or method will be used



Data Structures

Introduction to Data Structures

Primitive and Non-Primitive data Types:

Primitive Data types:

The basic data types:

- **Integer:** represents a number without decimal point. Ex. 12, 90
- **Float and Double:** represents a number with decimal point. Ex. 65.1, 90.0
- **Character:** represents a single character Ex. 'C', 'a'
- **String:** represents group of characters. Ex. "University of Kufa"
- **Boolean:** represents logical values either true or false.



Data Structures

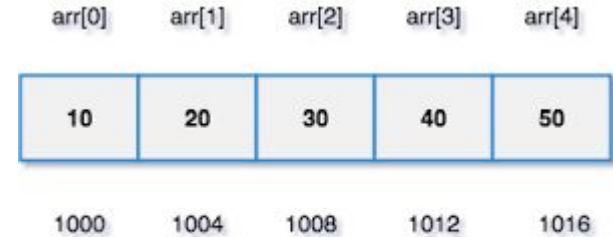
Introduction to Data Structures

Non-Primitive Data types:

Derived from primary data types and used to store group of values like:

Arrays:

Collection of data items of the same type.
Can be accessed via index.



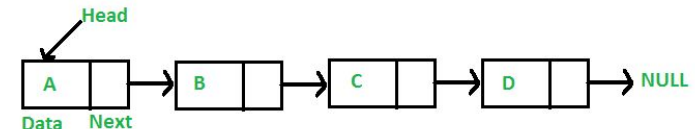
Structure:

Allows to combine data items of different types.
Can be accessed via a single address.

```
struct contact_information {  
    char firstname[100];  
    char lastname[200];  
    int homephone;  
    int mobilephone;  
};
```

linked list:

The elements are linked using pointers



Data Structures

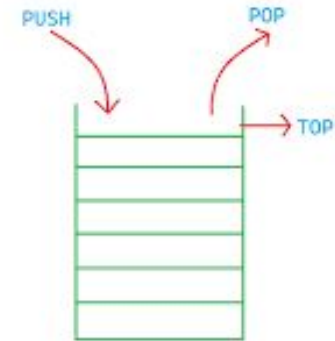
Introduction to Data Structures

Non-Primitive Data types:

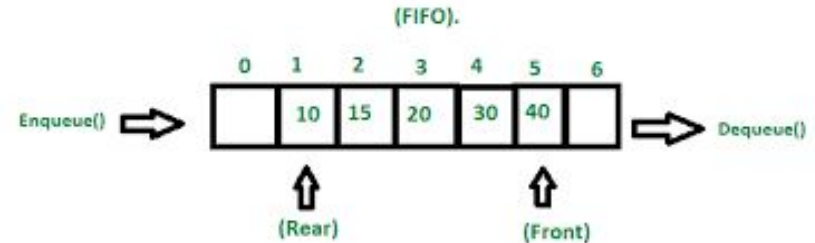
Derived from primary data types and used to store group of values like:

Stacks: collection of elements, based on the ***Last-In-First-Out*** with two main operations:

- **Push**, adds an element to the collection
- **Pop**, removes the most recently added element



Queue: based on the ***First-In-First-Out*** and has two ends called **front** and **rear**.



Data Structures

Introduction to Data Structures

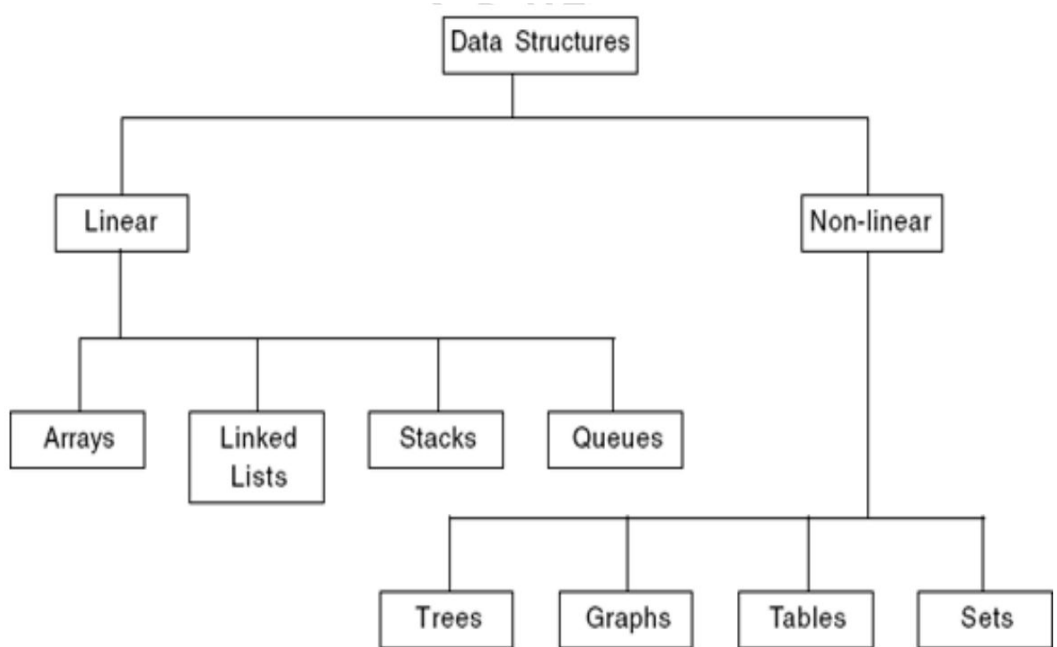
Types of data structures

Linear Data Structure:

Every element is linked with the next one sequentially.

Nonlinear data structure:

The element may attach to more than one element and not in sequence.



Data Structures

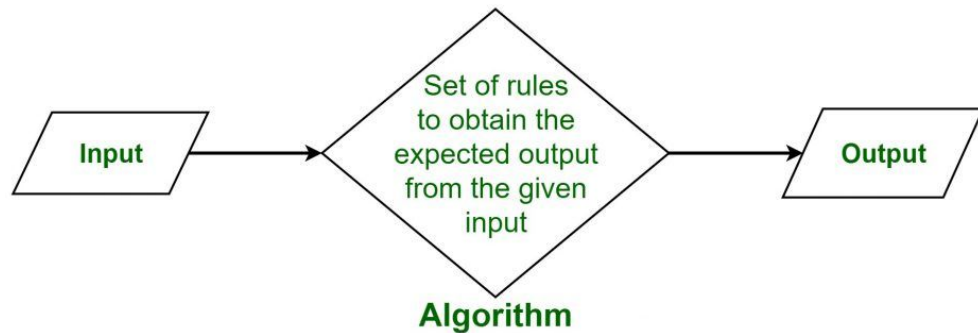
Algorithms

Algorithm: is a procedure or a sequence of steps that takes an input and transforms it into a target output. (Correctness and complexity)

Together, data structures and algorithms combine and allow programmers to build whatever computer programs wants to do.

Algorithms can be expressed as:

- natural languages,
- pseudocode,
- flowcharts,
- programming languages ..



Programming languages expresses algorithms in a form that can be executed by a computer, but are also used to define algorithms.

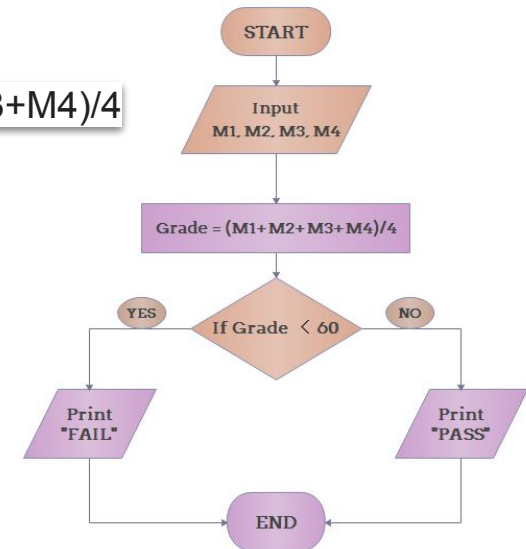
Data Structures

Algorithms

Typical steps in the development of algorithms:

1. Problem definition
2. Development
3. Specification
4. Design
5. Checking the correctness
6. Implementation
7. Testing
8. Documentation

1. Start
2. Input: M1, M2, M3, M4
3. $\text{Average} = (M1 + M2 + M3 + M4) / 4$
4. If Average < 60
5. Print "Fail"
6. Else
7. Print "Pass"
8. End



Data Structures

Software Development Life Cycle Models

Software Development Life Cycle is the application of standard practices to build software applications. It's typically divided into six to eight steps:

- Requirements gathering and analysis
- Design
- Implementation or coding
- Testing
- Deployment
- Maintenance

The purpose of SDLC is to deliver a high-quality product that is as the customer's requirement.



Data Structures

Software Development Life Cycle Models

Waterfall Model

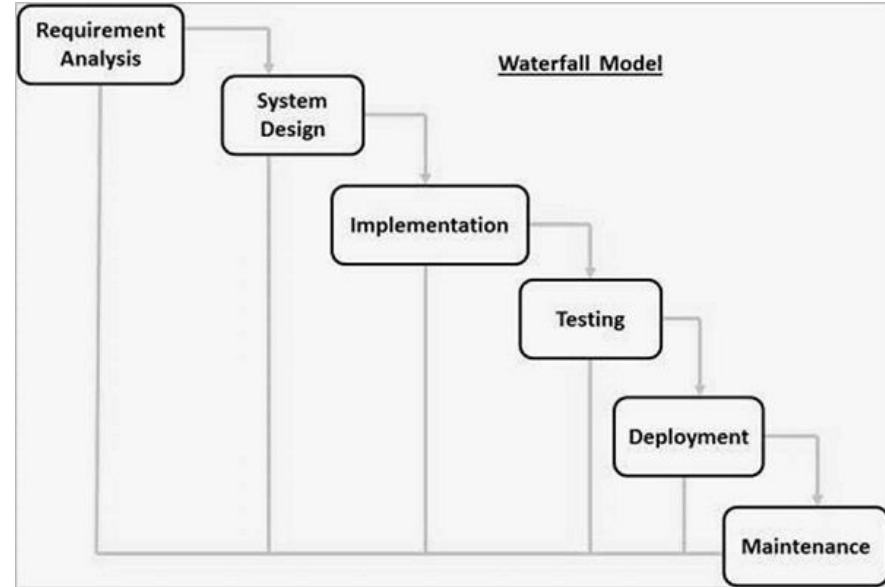
The process is divided into phases, where the outcome of one phase works as the input for the next phase sequentially.

Advantages:

- Simple and easy to understand and use.
- Good for smaller projects where requirements are very well understood.

Disadvantages:

- No working software until late during the life cycle.
- Not good for long projects.
- Cannot accommodate changing requirements.



Data Structures

Software Development Life Cycle Models

Iterative Model

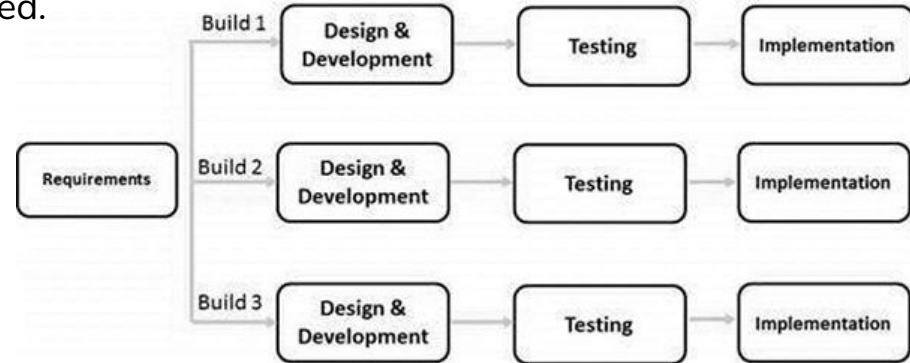
Starts with a simple implementation of a subset of the software requirements and iteratively enhances the evolving versions until the full system is implemented.

Advantages:

- With every increment, operational product is delivered.
- Less costly to change the requirements.
- Testing during smaller iteration is easy.
- Better suited for large projects.

Disadvantages:

- More resources may be required.
- Not suitable for smaller projects.



Data Structures

Software Development Life Cycle Models

Spiral Model

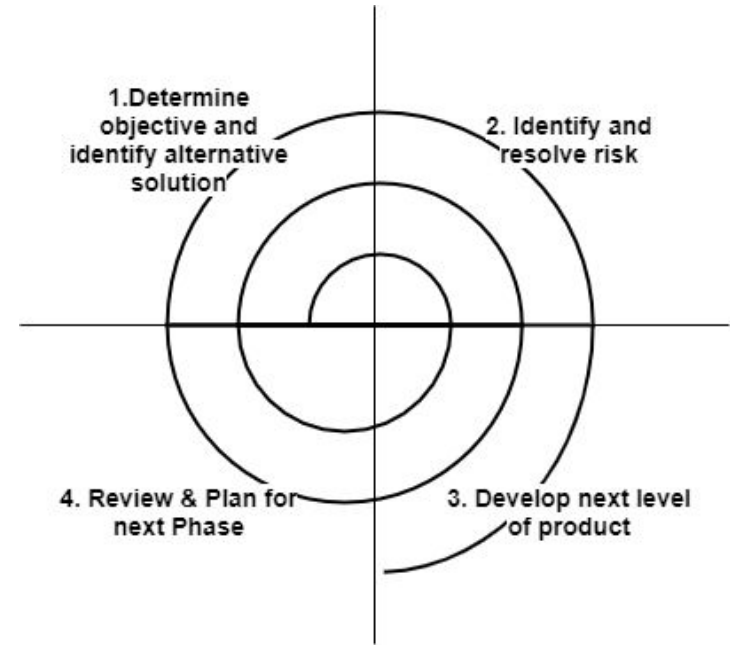
A software project repeatedly passes through **four** phases in iterations called Spirals.

Advantages:

- Changing requirements can be accommodated.
- Users see the system early.
- Development can be divided into smaller parts and the risky parts can be developed earlier for better risk management.

Disadvantages:

- Management is more complex.
- End of the project may not be known early.
- It could be expensive for small projects.



Data Structures

Software Development Life Cycle Models

V-Model - Design

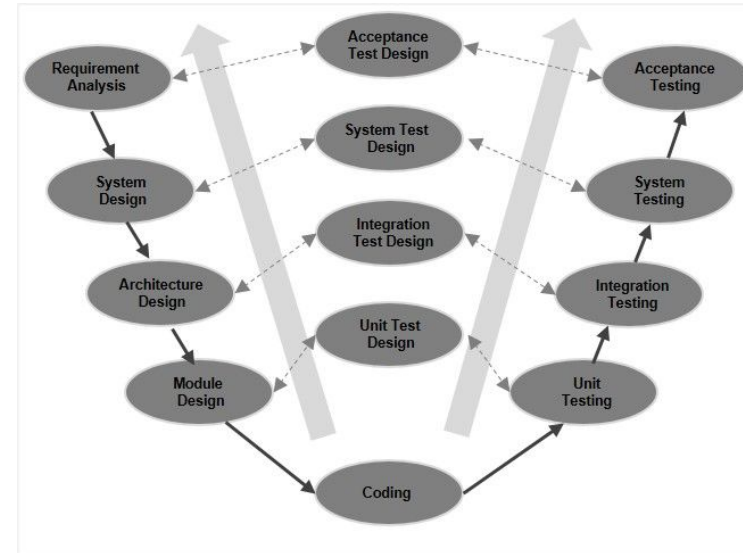
The testing phase for each development phase is planned in parallel (Verification phases on one side of the 'V' and Validation phases on the other side). Coding Phase joins the two sides of the V-Model.

Advantages:

- Phases are completed one at a time.
- Good for smaller projects where requirements are very well understood.
- Simple and easy to understand and use.

Disadvantages:

- Not good for complex projects.
- Not good for the projects where requirements are changing.
- It is difficult to go back / change during the testing stage.
- No working software until late during the life cycle.



Data Structures

Software Development Life Cycle Models

Agile Design

It break the product into small incremental builds.
These builds are provided in iterations.

Advantages:

- Suitable for fixed or changing requirements
- Delivers early partial working solutions.
- Easy to manage.

Disadvantages:

- If customer is not clear, team can be driven in the wrong direction.
- There is minimum documentation generated, so, transfer of technology to new team members may be quite challenging

